Week No. 6

Lab Manual: Information Security

Lab Title: Ciphers with OpenSSL

Lab Objectives

By the end of this lab students will be able to:

12. Understand and apply symmetric encryption using OpenSSL commands.
13. Perform encryption and decryption using AES, DES, and 3DES algorithms.
14. Generate RSA key pairs and use them for encryption and decryption.
15. Observe differences between symmetric and asymmetric approaches.

Learning Outcomes (CLOs & PLOs Mapping)

CLO	CLO Statement	Bloom's Level	PLO
1	Apply Information Security techniques into practice.	P3 (Apply)	PLO-5

Lab Overview

This lab introduces students to the practical use of OpenSSL for implementing and analyzing symmetric key encryption techniques. Students will learn how to encrypt and decrypt files or messages using various symmetric algorithms such as AES, DES, and 3DES through OpenSSL command-line utilities. The lab then adds the asymmetric algorithm RSA so that the two approaches can be compared.

How this manual works: read the concepts first, then run each command in order. Every command has a one-line summary, a breakdown of its options and a sample output. The sample outputs were produced by running the same commands, so yours should look very similar (random values, file times and sizes of keys will differ).

Before you begin

Use the Kali (or any Linux) terminal. Work in a new folder so the files stay together:

```
mkdir ~/openssl-lab && cd ~/openssl-lab
```

Then create the two test files used by this lab (see “Step 0” below).

Key Concepts Explained Simply

1. Symmetric vs Asymmetric Encryption

	Symmetric	Asymmetric
Keys	One secret key used for both encrypting and decrypting	A pair: a public key (shared) and a private key (secret)
Everyday picture	A house key: the same key locks and unlocks	A padlock anyone can snap shut (public key); only you hold the key that opens it (private key)

	Symmetric	Asymmetric
Speed	Very fast; good for large files	Much slower; only for small data
Main problem	How do both people safely share the secret key?	Slow, and limited in how much data it can encrypt
Examples in this lab	AES, DES, 3DES	RSA

2. AES, DES and 3DES (Symmetric Algorithms)

All three are **block ciphers**: they cut the data into fixed-size blocks and encrypt one block at a time.

Algorithm	Block size	Key size	Status today
AES (aes-256-cbc)	128 bits (16 bytes)	256 bits here (also 128, 192)	Current standard. Safe to use.
DES (des-cbc)	64 bits (8 bytes)	56 effective bits	Broken. Too short a key; can be brute-forced. Learning only.
3DES (des3)	64 bits (8 bytes)	Three DES keys (up to 168 bits)	Slow and being phased out. Learning only.

3DES in simple words: it runs DES three times (encrypt, decrypt, encrypt) with different keys to make the key harder to guess.

3. Modes, IV and Padding (what "CBC" means)

- **CBC (Cipher Block Chaining)** is the way blocks are linked. Each block is mixed with the previous ciphertext block before encryption, so equal plaintext blocks do not give equal ciphertext blocks.
- **IV (Initialization Vector)** is a random starting value for the chain. It makes the same message encrypt differently each time.
- **Padding:** the last block is filled up to the full block size. So the ciphertext is always a multiple of the block size (16 bytes for AES, 8 for DES/3DES).

4. Passphrase, Salt and the "Salted__" Header

In this lab you type a passphrase with **-k** (for example mysecretkey). OpenSSL does not use that text directly. It mixes it with a random 8-byte **salt** to build the real key and IV. The salt is saved at the start of the output file, after the 8 letters **Salted**__. That is why:

- Every encrypted file starts with 16 extra bytes (8 for the word Salted__, 8 for the salt).
- Encrypting the same file twice with the same passphrase gives different ciphertext (different salt).
- The passphrase alone is enough to decrypt: OpenSSL reads the salt from the file.

5. RSA in Simple Words

- You generate a **key pair**: a public key and a private key that are mathematically linked.
- Data locked with the **public key** can only be unlocked with the **private key**.
- A 2048-bit key means a 2048-bit number (the *modulus*) is used, which is 256 bytes. **Every RSA ciphertext is exactly 256 bytes long**, however short the message is.

- RSA can only encrypt data **smaller than the key**. With the padding OpenSSL uses by default, a 2048-bit key can encrypt at most **245 bytes**. Larger data gives an error (tested below).
- Keys are saved in **PEM** files: text files with a Base64 body between lines like -----BEGIN PUBLIC KEY-----.

6. Hybrid Cryptosystem

Real systems (HTTPS, email encryption) combine both: **AES encrypts the data** (fast) and **RSA encrypts only the small AES key** (safe sharing). You will try a small hybrid demo in Part 3.

7. How an OpenSSL Command Is Built

Every command has the shape: **openssl <tool> <options>**. The tool says *what* to do (enc = symmetric encryption, rand = random bytes, genpkey = make a key, rsa = work with RSA keys, pkeyutl = use a key to encrypt/decrypt) and the options say *how* (input file, output file, algorithm, key).

Option	Meaning
-in file	File to read (the input).
-out file	File to write (the output).
-d	Decrypt instead of encrypt (without -d it encrypts).
-k text	Passphrase to derive the key from.
-aes-256-cbc, -des-cbc, -des3	Which algorithm and mode to use.

Step 0: Prepare Your Lab Files

Check OpenSSL is installed

In one line: Prints the OpenSSL version.

Command

```
openssl version
```

What each part means

- Kali includes OpenSSL. Version 3.x is common, and it matters for DES (see Part 1).

Sample output (tested)

```
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

Your version number may be different.

Create the test files

In one line: echo writes text and > saves it into a file; wc -c counts the bytes.

Commands

```
echo "Meet me at the library at 5 PM. This is a secret message for the lab." >
plaintext.txt
echo "Hello RSA" > message.txt
cat plaintext.txt
wc -c plaintext.txt
```

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
70 plaintext.txt
```

plaintext.txt (70 bytes) is used for AES, DES, 3DES; message.txt is used for RSA. You can use your own text. Remember its size: you will compare it with the encrypted files later.

Useful OpenSSL Helpers: Random Bytes

Good keys must be unpredictable. **openssl rand** produces cryptographically random bytes.

1. Random key and IV as binary files

In one line: Writes 32 random bytes (a 256-bit key) and 16 random bytes (a 128-bit IV) into files.

Command

```
# 32 bytes (256-bit) key in binary
openssl rand -out key.bin 32
# 16-byte IV (128-bit)
openssl rand -out iv.bin 16
ls -l key.bin iv.bin
```

What each part means

- **rand** is the tool; **-out file** names the output file; the last number is how many bytes to create.
- 1 byte = 8 bits, so 32 bytes = 256 bits and 16 bytes = 128 bits.

Sample output (tested)

```
-rw-r--r-- 1 root root 16 Nov  5 05:11 iv.bin
-rw-r--r-- 1 root root 32 Nov  5 05:11 key.bin
```

The files hold raw bytes, so cat shows unreadable symbols. That is normal.

2. Random key as readable hex

In one line: Prints 32 random bytes as text (hexadecimal) on the screen.

Command

```
# Hex outputs (human-readable)
openssl rand -hex 32     # prints 64 hex chars -> 256 bits
```

What each part means

- **-hex** shows each byte as two hex characters (0-9, a-f).
- 32 bytes × 2 characters = **64 characters**, which equals 256 bits. Count them in the sample below.
- The value is different every time you run it.

Sample output (screenshot from the lab)

```
localhost:~# openssl rand -hex 32
5edaee2db6fba77b65076ef59205e82c288bc70e0c37edd80c7ce1fbe05640d2
localhost:~#
```

Part 1: Symmetric Encryption using OpenSSL

The pattern is the same for every algorithm: **encrypt** with the algorithm name, then **decrypt** with the same command plus **-d** and swapped input/output files.

1. AES Encryption and Decryption

Encrypt a file using AES-256-CBC

In one line: Encrypts plaintext.txt with a 256-bit AES key built from the passphrase mysecretkey.

Command

```
# Encrypt file using AES-256-CBC
openssl enc -aes-256-cbc -in plaintext.txt -out ciphertext.enc -k mysecretkey
```

What each part means

- **enc** = symmetric encryption tool.
- **-aes-256-cbc** = AES, 256-bit key, CBC mode.
- **-in plaintext.txt** = the file to protect. **-out ciphertext.enc** = where the encrypted result is written.
- **-k mysecretkey** = the passphrase (choose your own in the lab).

Sample output (tested)

```
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
(no other output = success)
```

About the warning: it is only a notice that the old passphrase-to-key method is used. The command still works. To remove it, add **-pbkdf2** to both the encrypt and decrypt commands (see “Better practice” below).

> **Security tip**
> Typing a passphrase after **-k** puts it in your shell history and in the process list. That is acceptable for a lab but not for real secrets.

Decrypt the file

In one line: Reverses the encryption: -d tells OpenSSL to decrypt.

Command

```
# Decrypt file
openssl enc -aes-256-cbc -d -in ciphertext.enc -out decrypted.txt -k mysecretkey
cat decrypted.txt
diff plaintext.txt decrypted.txt && echo SAME
```

What each part means

- **-d** = decrypt. Input is now the encrypted file and output is the recovered file.
- The algorithm and passphrase **must match** the ones used to encrypt.
- **diff** compares two files; no output from diff means they are identical.

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
SAME
```

2. DES Encryption and Decryption

Encrypt using DES

In one line: Encrypts the file with the old DES algorithm (56-bit key). For learning only.

Command

```
# Encrypt using DES
openssl enc -des-cbc -in plaintext.txt -out ciphertext_des.enc -k deskey
```

What each part means

- **-des-cbc** = DES in CBC mode.
- All other options are the same as for AES.

Sample output (tested)

```
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
Error setting cipher DES-CBC
```

OpenSSL 3 needs a flag for DES (tested)

DES is considered unsafe, so OpenSSL 3.x moved it into the **legacy provider**, and the plain command above fails with "Error setting cipher DES-CBC". If you see this, load the legacy provider by adding **-provider legacy -provider default**:

```
openssl enc -des-cbc -provider legacy -provider default -in plaintext.txt -out
ciphertext_des.enc -k deskey
```

Working command (OpenSSL 3.x):

```
openssl enc -des-cbc -provider legacy -provider default -in plaintext.txt -out
ciphertext_des.enc -k deskey
```

If your OpenSSL is older than 3.0, the original command without the extra flags works as it is.

Decrypt DES

In one line: Decrypts the DES file (same extra flags if needed).

Command

```
# Decrypt DES
openssl enc -des-cbc -d -provider legacy -provider default -in ciphertext_des.enc -out
decrypted_des.txt -k deskey
cat decrypted_des.txt
```

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
```

3. 3DES Encryption and Decryption

Encrypt using 3DES

In one line: Encrypts the file by running DES three times (key derived from the passphrase tripledes).

Command

```
# Encrypt using 3DES
openssl enc -des3 -in plaintext.txt -out ciphertext_3des.enc -k tripledes
```

What each part means

- **-des3** = Triple DES (DES-EDE3-CBC). It worked without extra flags in testing.

Sample output (tested)

```
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
(no other output = success)
```

Decrypt 3DES

In one line: Decrypts the 3DES file.

Command

```
# Decrypt 3DES
openssl enc -des3 -d -in ciphertext_3des.enc -out decrypted_3des.txt -k tripledes
cat decrypted_3des.txt
```

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
```

Better Practice (Optional)

These two variations were also tested. They are not required for the lab but show how real systems work.

A. Stronger passphrase processing: -pbkdf2

In one line: Same AES encryption, but with a modern key-derivation method; the warning disappears.

Command

```
openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -in plaintext.txt -out c3.enc -k
mysecretkey
openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -d -in c3.enc -k mysecretkey
```

What each part means

- **-pbkdf2** uses a modern method to turn the passphrase into a key; **-iter 100000** repeats it 100,000 times to slow down guessing.
- Use the **same** -pbkdf2 and -iter values for decryption.

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
```

B. Use the random key and IV files directly

In one line: Skips the passphrase and uses the random key.bin and iv.bin created earlier (capital -K and -iv take hex values).

Command

```
K=$(xxd -p -c 64 key.bin)
IV=$(xxd -p -c 32 iv.bin)
openssl enc -aes-256-cbc -K $K -iv $IV -in plaintext.txt -out c4.enc
openssl enc -aes-256-cbc -d -K $K -iv $IV -in c4.enc
```

What each part means

- **xxd -p** converts the binary file to hex text; the result is stored in the variables K and IV.
- **-K** (capital) = the key in hex; **-iv** = the IV in hex. No salt is used, so the file has no Salted__ header.

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
```

Part 2: Asymmetric Encryption (RSA)

1. Generate RSA Key Pair

Generate the private key (2048 bits)

In one line: Creates a new 2048-bit RSA private key and saves it as private.pem.

Command

```
# Generate private key (2048 bits)
openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048
```

What each part means

- **genpkey** = generate a private key; **-algorithm RSA** = make an RSA key.
- **-out private.pem** = file to save it in.
- **-pkeyopt rsa_keygen_bits:2048** = key size 2048 bits.

Sample output (tested)

```
..+++++++++++++++++++*...+++++++++++++++++++*
writing RSA key
```

The dots and plus signs are a progress display while OpenSSL searches for large prime numbers; they differ every time and it takes a second or two.

Extract the public key

In one line: Reads the private key and writes the matching public key into public.pem.

Command

```
# Extract public key from private key
openssl rsa -pubout -in private.pem -out public.pem
head -1 private.pem public.pem
```

What each part means

- **rsa** = RSA key tool; **-pubout** = output the public part; **-in** = key to read; **-out** = file to write.

Sample output (tested)

```
writing RSA key
==> private.pem <==
-----BEGIN PRIVATE KEY-----

==> public.pem <==
-----BEGIN PUBLIC KEY-----
```

Important: keep **private.pem** secret (never send or share it). **public.pem** can be given to anyone. In the test the private key file was about 1.7 KB and the public key about 0.5 KB.

2. Encrypt and Decrypt using RSA

Encrypt with the public key

In one line: Locks message.txt with public.pem so only the private key can open it.

Command

```
# Encrypt with public key
openssl rsautl -encrypt -inkey public.pem -pubin -in message.txt -out message.enc
ls -l message.enc
```

What each part means

- **rsautl** = RSA utility (the original tool).
- **-encrypt** = encrypt. **-inkey public.pem** = the key file to use. **-pubin** = the key is a public key.
- **-in / -out** = input and output files.

Sample output (tested)

```
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
-rw-r--r-- 1 root root 256 Nov  5 05:12 message.enc
```

Note the size: the file is exactly **256 bytes** even though message.txt has only 10 bytes.

Decrypt with the private key

In one line: Unlocks message.enc with private.pem and writes the original message.

Command

```
# Decrypt with private key
openssl rsautl -decrypt -inkey private.pem -in message.enc -out message_decrypted.txt
cat message_decrypted.txt
```

What each part means

- **-decrypt** = decrypt. The key now is the **private** key (no -pubin).

Sample output (tested)

```
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
Hello RSA
```

rsautl is deprecated in OpenSSL 3 (tested)

The rsautl commands above still work but print the deprecation message. The modern replacement is **pkeyutl** and gives the same result (tested):

```
openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out message.enc
openssl pkeyutl -decrypt -inkey private.pem -in message.enc -out message_decrypted.txt
```

What happens if the message is too big? (Demonstration)

In one line: Tries to RSA-encrypt a 406-byte file to show the size limit.

Commands

```
head -c 300 /dev/urandom | base64 > big.txt
wc -c big.txt
openssl pkeyutl -encrypt -pubin -inkey public.pem -in big.txt -out big.enc
```

Sample output (tested)

```
406 big.txt
Public Key operation error
error:0200006E:rsa routines:ossl_rsa_padding_add_PKCS1_type_2_ex:data too large for key
size
```

Tested limit with this key: a **245-byte** file encrypts fine, a **246-byte** file fails. This is why RSA cannot encrypt large files directly (Lab Question 2).

3. View Key Information

Display key details

In one line: Prints the numbers inside the private key in readable form.

Command

```
# Display key details
openssl rsa -in private.pem -text -noout
```

What each part means

- **-text** = show the details as text; **-noout** = do not print the PEM block again.

Sample output (tested)

```
Private-Key: (2048 bit, 2 primes)
modulus:
    00:95:08:b8:2a:40:f9:7d:1c:4b:41:21:97:52:cb:
    ... (many more lines)
publicExponent: 65537 (0x10001)
privateExponent:  ...
prime1:  ...
prime2:  ...
exponent1:  ...
exponent2:  ...
coefficient:  ...
```

Reading the output: the **modulus** is the big 2048-bit number shared by both keys. The **publicExponent** (65537) is the small number in the public key. The **privateExponent** is the secret part. **prime1** and **prime2** are the two secret prime numbers whose product is the modulus; whoever learns them can rebuild the private key.

Part 3: Verification and Comparison

1. Compare Ciphertext Sizes

In one line: ls -l lists files with their sizes in bytes (5th column).

Command

```
ls -l plaintext.txt ciphertext.enc ciphertext_des.enc ciphertext_3des.enc message.enc
```

Measured sizes for the 70-byte plaintext.txt (RSA: any short message gave the same result):

File	Size (bytes)	Why this size
plaintext.txt	70	Original text.
AES-256-CBC (ciphertext.enc)	96	70 padded to 80 (next multiple of 16) + 16-byte Salted__ header.
DES-CBC (ciphertext_des.enc)	88	70 padded to 72 (next multiple of 8) + 16-byte header.
3DES (ciphertext_3des.enc)	88	Same block size as DES: 72 + 16.
RSA 2048 (message.enc)	256	Always the key size (2048 bits = 256 bytes).

2. Inspect the Files: file and hexdump

file

In one line: Guesses what kind of data a file contains.

Command

```
file ciphertext.enc ciphertext_des.enc message.enc
```

Sample output (tested)

```
ciphertext.enc:     openssl enc'd data with salted password
ciphertext_des.enc: openssl enc'd data with salted password
message.enc:        data
```

OpenSSL recognises the Salted__ header of the symmetric files; the RSA file has no header, so it is just “data”.

hexdump

In one line: Shows a file's bytes in hexadecimal (left) and as text (right); -C = “canonical” layout and | head = show only the first 10 lines.

Command

```
hexdump -C ciphertext.enc | head
```

What each part means

- Each line starts with the byte position, then 16 bytes in hex, then the readable characters between | | bars.

Sample output (tested)

```
00000000  53 61 6c 74 65 64 5f 5f  a1 be f1 99 23 ac 05 5b  |Salted__....#..[|
00000010  4d 1d a8 76 be ae 7d 98  1f 65 83 90 11 b3 f1 d0  |M..v..}..e......|
00000020  96 30 37 9c 91 0e d5 2c  2a bb ca 70 7a 90 35 62  |.07....,*..pz.5b|
```

Observe: the first 8 bytes spell **Salted__** (53 61 6c 74 65 64 5f 5f in hex). Everything after the salt looks random; no part of the plaintext is visible. Run the same command on message.enc and notice there is **no** Salted__ header. If hexdump is missing, install it with: sudo apt install bsdextrautils

3. Discussion Points

- **Compare the ciphertext files.** AES, DES and 3DES outputs are a little longer than the plaintext (padding + 16-byte header). RSA output is always 256 bytes, longer than the symmetric output for short messages.
- **RSA is used for key exchange, not bulk data.** It is slow and limited to 245 bytes per operation (with a 2048-bit key), so it is used only to protect a small secret key. The actual data is encrypted with a fast symmetric cipher such as AES.

4. Mini Hybrid Encryption Demo (Tested)

This shows how the two approaches work together. **Sender** (has public.pem):

```
openssl rand -hex 32 > session.key
openssl enc -aes-256-cbc -pbkdf2 -in plaintext.txt -out data.enc -pass file:session.key
openssl pkeyutl -encrypt -pubin -inkey public.pem -in session.key -out session.key.enc
```

Sender sends **data.enc** and **session.key.enc**. **Receiver** (has private.pem):

```
openssl pkeyutl -decrypt -inkey private.pem -in session.key.enc -out
session_recovered.key
openssl enc -aes-256-cbc -pbkdf2 -d -in data.enc -pass file:session_recovered.key
```

Sample output (tested)

```
Meet me at the library at 5 PM. This is a secret message for the lab.
```

Step by step: (1) create a random session key, (2) encrypt the data with AES using that key, (3) encrypt the small session key with RSA. The receiver reverses it: RSA-decrypt the session key, then AES-decrypt the data. Sizes in the test: session.key 65 bytes, session.key.enc 256 bytes, data.enc 96 bytes.

Lab Questions

Q1. What are the main differences between symmetric and asymmetric encryption in terms of speed and security?

Answer:

Answer:

Answer:

Q2. Why is RSA not suitable for encrypting large files directly?

Answer:

Answer:

Answer:

Q3. How can symmetric and asymmetric encryption be combined in a hybrid cryptosystem?

Answer:

Answer:

Answer:

Q4. Observe ciphertext sizes. Why does RSA output appear longer than AES ciphertext for the same plaintext?

Answer:

Answer:

Answer:

Hints: *Q1: how many keys, and how is the key shared? Q2: look at the "data too large for key size" error. Q3: see the mini hybrid demo. Q4: compare the sizes table in Part 3.*

Troubleshooting

Message or problem	Meaning	Fix
* WARNING : deprecated key derivation us	Old passphrase-to-key method (harmless).	Ignore it, or add -pbkdf2 to both encrypt and decrypt.
Error setting cipher DES-CBC	OpenSSL 3 treats DES as legacy.	Add -provider legacy -provider default.
bad decrypt	Wrong passphrase, or different algorithm/options than used to encrypt.	Use the same algorithm, passphrase and options (-pbkdf2 etc.) as for encryption.
Public Key operation error ... data too large for key size	RSA input is bigger than the key allows (245 bytes for 2048 bits).	Encrypt a smaller file, or use hybrid encryption.
The command rsautl was deprecated	OpenSSL 3 replaced rsautl.	It still works; or use openssl pkeyutl.
hexdump: command not found	Package missing.	sudo apt install bsdextrautils
No such file or directory	The input file is not in the current folder.	Check with ls and cd into your lab folder.

Command Quick Reference

Command	One-line summary
openssl rand -out key.bin 32	Create 32 random bytes (256-bit key) in a file.
openssl rand -hex 32	Print 32 random bytes as 64 hex characters.
openssl enc -aes-256-cbc -in f -out g -k pw	Encrypt file f with AES-256-CBC into g.
openssl enc -aes-256-cbc -d -in g -out f2 -k pw	Decrypt g back into f2 (-d = decrypt).
openssl enc -des-cbc ...	Same with DES (OpenSSL 3: add -provider legacy -provider default).
openssl enc -des3 ...	Same with Triple DES.
openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048	Generate a 2048-bit RSA private key.
openssl rsa -pubout -in private.pem -out public.pem	Extract the public key from the private key.
openssl rsautl -encrypt -inkey public.pem -pubin -in m -out e	Encrypt a small file with the public key (modern: pkeyutl).
openssl rsautl -decrypt -inkey private.pem -in e -out m2	Decrypt it with the private key.
openssl rsa -in private.pem -text -noout	Show the numbers inside the private key.
file x	Say what kind of file x is.
hexdump -C x | head	Show the first bytes of x in hex and text.
ls -l / wc -c	List files with sizes / count the bytes in a file.